Neighbourhood operators

Nighborhood operators

- Neighborhood operators are image processing operations where the new value of a pixel depends on the values of nearby pixels (its neighborhood) not just on itself.

What is a “neighborhood”?

- Take any pixel in an image. The pixels around it form its neighborhood.

Common neighborhoods

3×3 neighborhood

a b c
d e f
g h i

- e → pixel being updated
- a…i → neighbors used to compute the new value

Why do we need neighborhood operators?

- Point operators change **brightness only**.

But many important tasks need **spatial context**:

- Remove noise → need nearby pixels
- Detect edges → need intensity differences
- Blur or sharpen → need local averaging or contrast
- Detect shapes → need local structure

All these require **neighborhood information**.

Classes of Neighborhood Operators

- Linear Neighborhood Operators
- Non-Linear Neighborhood Operators

Linear Neighborhood Operators

General formula

$$g(x,y) = \sum_{i,j} h(i,j)\, f(x+i, y+j)$$

- f: input image
- g: output image
- h: kernel (weights)

- **Examples**
- Mean filter
- Gaussian filter
- Sobel filter
- Laplacian filter

Non-Linear Neighborhood Operators

- General Form

For a neighborhood $\mathcal{N}(x, y)$ around pixel (x, y):

$$g(x, y) = \Phi\big(\{f(i, j) \mid (i, j) \in \mathcal{N}(x, y)\}\big)$$

Where:

- $f(i, j)$ → input pixel values in neighborhood
- $g(x, y)$ → output pixel
- $\Phi(\cdot)$ → **non-linear function**

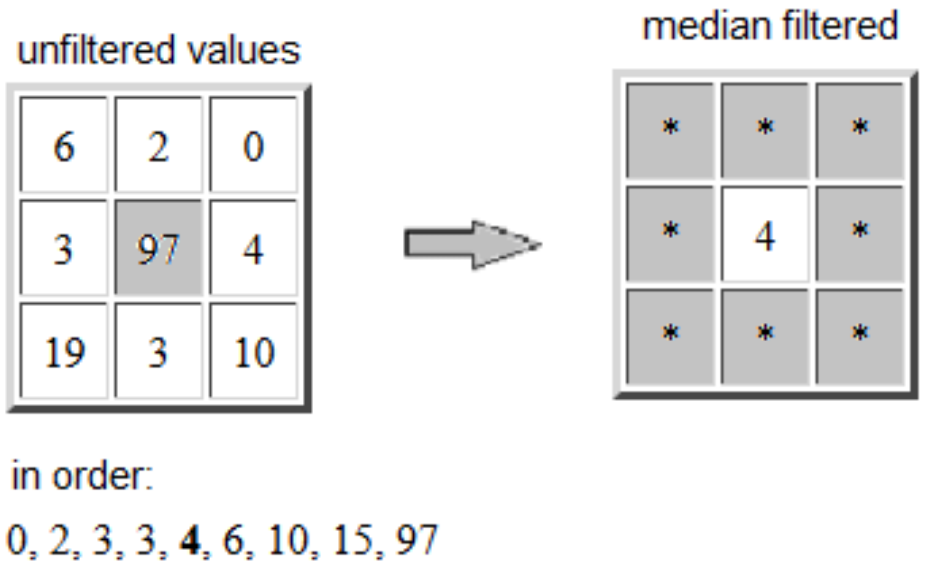
Non-Linear Neighborhood Operators

- Median filter
- Morphological operations (erosion, dilation)
- Adaptive thresholding

Median Filter

- **Purpose**
  - Remove salt-and-pepper **noise**
  - Preserve edges better than mean filter

$$g(x,y) = \text{median}\{f(i,j) \mid (i,j) \in \mathcal{N}(x,y)\}$$



in order:

0, 2, 3, 3, **4**, 6, 10, 15, 97

Min Filter

Purpose

- Remove **bright noise**
- Shrink bright regions

$$g(x,y) = \min\{f(i,j) \mid (i,j) \in \mathcal{N}(x,y)\}$$

Max Filter

Purpose

- Remove **dark noise**
- Expand bright regions

$$g(x,y) = \max\{f(i,j) \mid (i,j) \in \mathcal{N}(x,y)\}$$

Median Filter

Original Image

with Median Filter

Morphological Operators

*Morphology works mainly on **binary images** (0 and 1).*

Erosion: For every pixel, erosion **looks at its neighbors** and assigns the **smallest intensity value** to the output pixel.

$$g(x,y) = \min_{(i,j)\in B} f(x+i, y+j)$$

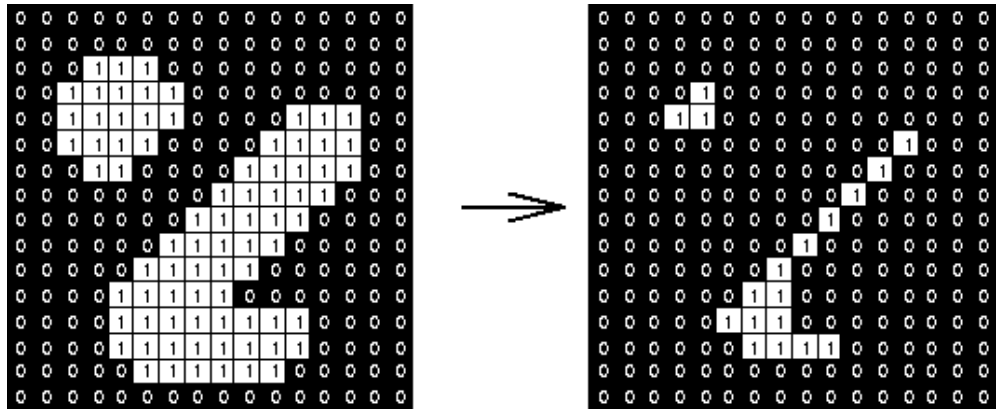
$f(x,y)$ → Input image

$g(x,y)$ → Output image

B → Structuring element (neighborhood)

$(i,j) \in B$ → All pixels inside the kernel

min → Take the **minimum value** in the neighborhood



Before erosion:

0 1 1 1 0
1 1 1 1 1
1 1 1 1 1

After erosion:

0 0 1 0 0
0 1 1 1 0
0 0 1 0 0

- Shrinks objects
- Removes small bright regions

At each pixel (x,y), look at all pixels in its neighborhood B and **take the minimum value**.

Morphological Operators

Grayscale Erosion

- Dark pixels dominate
- Bright regions become thinner

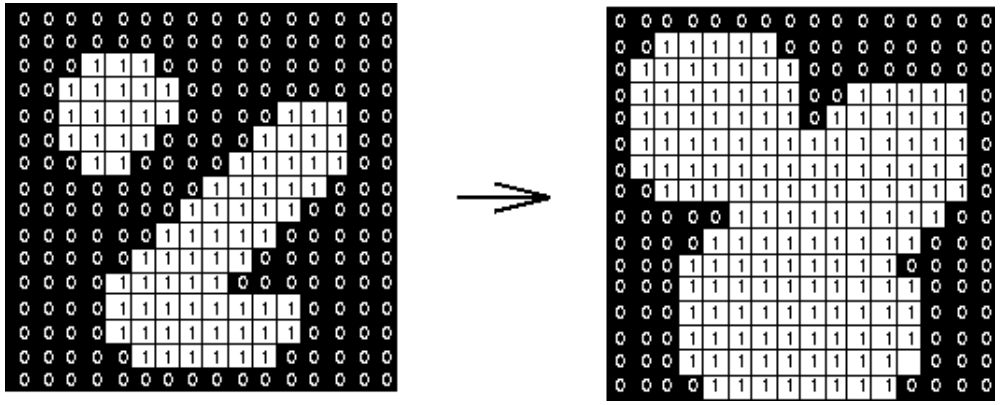
Useful for:

- Removing **salt noise**
- Separating connected objects

Morphological Operators

- **Dilation**: It **spreads bright regions outward**

$$g(x,y) = \max_{(i,j)\in B} f(x+i, y+j)$$



- Takes the **maximum value** in the neighborhood
- If **any neighbor is bright**, output becomes bright

Before dilation:

0 0 1 0 0
0 1 1 1 0
0 0 1 0 0

After dilation:

0 1 1 1 0
1 1 1 1 1
0 1 1 1 0

- Bright pixels **grow**
- Gaps and holes **fill**
- Thin objects **become thicker**

At each pixel (x, y), look at all pixels in its neighborhood and **take the maximum value**.

Erosion vs Dilation

Feature	Erosion	Dilation
Mathematical op	min	max
Object size	Shrinks	Expands
Noise handling	Removes bright noise	Removes dark gaps
Boundary	Moves inward	Moves outward

Original Image

Erosion

Dilation

Applications of Morphological Operators

- Document scanning
- License plate recognition
- Medical images with uneven contrast
- Text extraction from natural scenes

Adaptive Thresholding

- **Adaptive thresholding** converts a **grayscale image into a binary image**, but **the threshold changes from pixel to pixel**, based on **local neighborhood statistics**.

$$g(x,y) = \begin{cases} 1, & \text{if } f(x,y) > \mu_N - C \\ 0, & \text{otherwise} \end{cases}$$

Given:

- Pixel value:

$$f(x,y) = 140$$

- Local neighborhood mean:

$$\mu_N = 130$$

- Constant:

$$C = 5$$

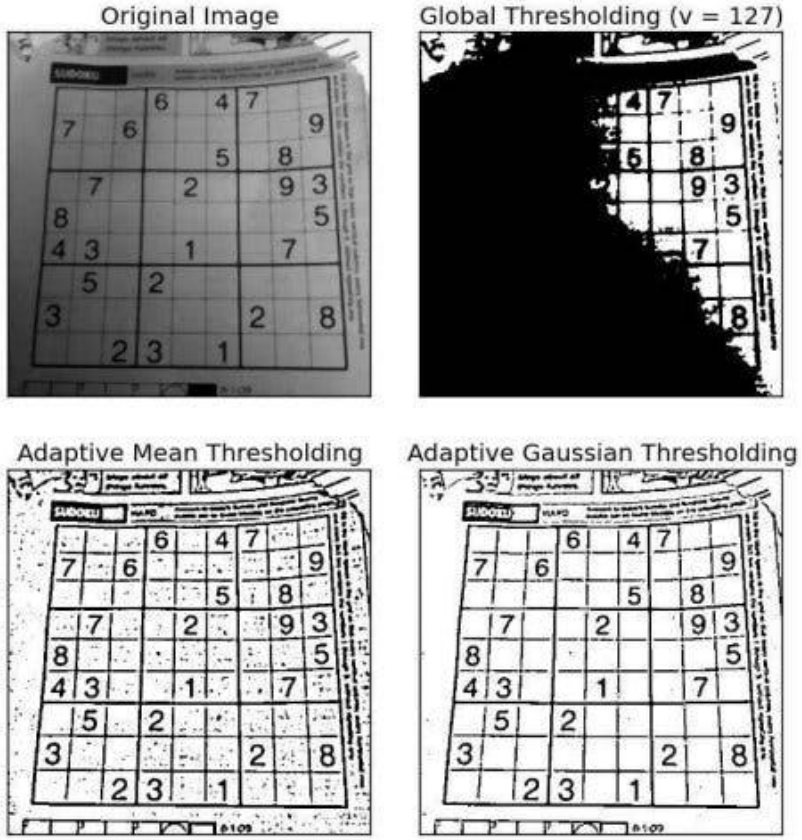
Step 1: Compute threshold

$$T = \mu_N - C = 130 - 5 = 125$$

Step 2: Compare

$$140 > 125 \Rightarrow g(x,y) = 1$$

Pixel becomes **white (foreground)**



Another Case:

- $f(x,y) = 120$

$$120 < 125 \Rightarrow g(x,y) = 0$$

Pixel becomes **black (background)**